

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

KHOA CÔNG NGHỆ THÔNG TIN

----- oOo -----



BÀI TẬP 03

HỌC MÁY CƠ BẢN

TS. HOÀNG QUỐC VIỆT

Hưng Yên – Tháng 09 năm 2026

BÀI TẬP 03: MÔ HÌNH PHÂN LOẠI

A. MỤC TIÊU

Sau bài tập này người học có thể:

- Hiểu được các mô hình phân loại trong học máy: nhị phân và đa lớp.
- Xây dựng được mạng nơ-ron nhân tạo cho bài toán phân loại nhị phân và đa lớp.
- Thực hiện đào tạo và đánh giá mạng nơ-ron đã xây dựng.

B. MÔI TRƯỜNG LÀM VIỆC

Với đặc thù của môn Học máy cơ bản, mục này sẽ liệt kê một số công cụ sử dụng để làm bài tập. Trong bài tập này, người học cần kiểm tra và chắc chắn các phần mềm sau trên máy tính còn hoạt động tốt:

1. Jupyter notebooks.
2. Google colab.

C. TÀI NGUYÊN THAM CHIẾU

Để hoàn thành tốt bài tập này người học nên tham khảo các tài nguyên sau:

STT	Tên tài nguyên	Mô tả tài nguyên
1	sepsis.csv	File dataset.

D. YÊU CẦU

Bài 1. Xây dựng mạng nơ-ron cho bài toán dự đoán khả năng một bệnh nhân mắc bệnh nhiễm trùng huyết có tử vong hay không? Các yêu cầu (mỗi công việc thực hiện trên 1 cell của Jupyter notebook):

- Yêu cầu 1: Đọc dữ liệu từ file *sepsis.csv*. Chia dữ liệu random thành 2 phần: training (75%) và phần còn lại cho testing (25%).
- Yêu cầu 2: Xây dựng mạng nơ-ron có tên `model_sepsis` có 4 lớp ẩn. Lớp ẩn 1 và 3 sử dụng **Conv1D**, lớp ẩn 2 và 4 gồm 16 và 32 units.
- Yêu cầu 3: Lựa chọn hàm loss và tối ưu hoá cho mô hình đó.

- Yêu cầu 4: Tiến thành training mạng nơ-ron đã xây dựng trên tập dữ liệu đã được xử lý ở bước 1 với 50 epochs, batch size = 64, 15% cho tập validation, và lưu lại các model đã huấn luyện.
- Yêu cầu 5: Đánh giá model đã được huấn luyện trên tập testing.
- Yêu cầu 6: Huấn luyện lại mô hình 100 epochs và đánh giá lại trên tập testing. Xem kết quả có cải thiện hay không?

Bài 2. Xây dựng mạng nơ-ron cho bài toán phân loại ảnh MNIST (phân loại 10 lớp):

- Yêu cầu 1: Load tập dữ liệu ảnh MNIST và chia tập dữ liệu này làm 2 phần: training và testing.
- Yêu cầu 2: Hiển thị 25 ảnh ngẫu nhiên từ tập training (hiển thị ảnh màu) và 25 ảnh ngẫu nhiên từ tập testing (hiển thị ảnh gray). Hiển thị với kích thước 5 hàng $\times$ 5 cột.
- Yêu cầu 3: Xây dựng mạng nơ-ron gồm 6 lớp ẩn với số Unit lần lượt là 8, 16, 64, 128, 32, 16.
- Yêu cầu 4: Tiến thành training mạng nơ-ron đã xây dựng trên tập dữ liệu đã được xử lý 50 epochs, 10% làm tập validation, batch size = 16 và lưu lại các model. Yêu cầu hiển thị loss sau mỗi 5 epochs.
- Yêu cầu 5: Đánh giá model đã được training trên tập testing.
- Yêu cầu 6: Huấn luyện lại model 100 epochs và đánh giá lại model đã được training trên tập testing. So sánh kết quả 2 lần huấn luyện.

Bài 3 (*). Triển khai model đã được huấn luyện:

- Yêu cầu: Triển khai model đã được huấn luyện ở bài 2 để xây một ứng dụng Winform trên Pycharm nhận dạng chữ số viết tay (0-9).

E. QUY ĐỊNH FILE MÃ NGUỒN

- Người học tạo thư mục *Code* lưu trong máy tính. Sau đó copy file dataset vào thư mục đó.
- File code bài 1 có định dạng: **Practice03Exercise01_HoVaTen.ipynb**. Ví dụ:
 - Practice03Exercise01_NguyenVanA.ipynb.
 - Practice03Exercise01_LeAnhB.ipynb...

- File code bài 2 có định dạng: **Practice03Exercise02_HoVaTen.ipynb**.
- Sinh viên cần upload các files kết quả lên link được cung cấp.
- Code tham khảo bài 2:
 - Hiện thị Dataset:

```
# BƯỚC 1 (YÊU CẦU 1): XỬ LÝ DỮ LIỆU MNIST
# MNIST chứa 70.000 ảnh chữ viết tay với kích thước 28*28

import keras
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

# 1.1. Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Hiện thị Dataset
for i in range(20):
    plt.subplot(4, 5, 1 + i)
    plt.axis('off')
    plt.imshow(X_train[i])
    # plt.imshow(x_train[i], cmap='gray')
plt.show()

# 1.2. Reduce pixel values: we need to convert data type to float for computation.
X_train = X_train.astype('float32')/255
X_test = X_test.astype('float32')/255

# 1.3. Normalize the data: Reshape x_train, x_test
X_train = X_train.reshape(X_train.shape[0], 784)
X_test = X_test.reshape(X_test.shape[0], 784)

# 1.4. One hot encoding.
y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)

# Finished
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
print("Finished!")
```

- Xây dựng mạng nơ-ron:

```
# XÂY DỰNG MẠNG NƠ RON
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.models import Sequential

# 2.1. Xây dựng mạng nơ-ron với 3 lớp ẩn
def model_MINIST():
    # Khởi tạo mạng
    model = Sequential()

    # Xây dựng mạng: 1 Input + 3 Hidden + 1 Output
    model.add(Dense(8, input_dim=6, activation='relu')) # => Sai 1 chỗ
    model.add(Dense(16, activation='relu'))
    model.add(Dense(4, activation='relu'))
    model.add(Dense(2, activation='softmax')) # Sai 1 chỗ

    # Return
    return model

# 2.2 Chọn model
model = model_MINIST()

# 2.3. Hiển thị thông tin model (tùy chọn)
model.summary()
```

○ **Hàm mất mát và tối ưu hoá:**

```
# LỰA CHỌN HÀM MẤT MÁT VÀ TỐI ƯU HOÁ
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy']) #
Hoặc
# model.compile(loss='categorical_crossentropy',
optimizer=keras.optimizers.Adam(learning_rate=0.0002), metrics=['binary_accuracy'])
```

- **Hướng dẫn bài 3.** Tham khảo code.